33

基于事件的并发（进阶）

到目前为止，我们讨论并发（concurrency）时，仿佛构建并发应用程序的唯一方法就是使用线程（thread）。和生活中的许多事情一样，这并不完全正确。具体来说，GUI 应用程序 [O96] 和某些类型的互联网服务器 [PDZ99] 中，常常采用另一种风格的并发编程（concurrent programming）。这种风格叫作**基于事件的并发**（event-based concurrency），在一些现代系统中很流行，包括 **node.js** [N13] 这类服务器端框架，但它的根源在 C/UNIX 系统中，下面就来讨论。

基于事件的并发（event-based concurrency）要解决的问题有两方面。一方面，在多线程（multi-threaded）应用程序中正确地管理并发可能很有挑战性；我们已经讨论过，可能会出现漏加锁（missing lock）、死锁（deadlock）以及其他令人头疼的问题。另一方面，在多线程应用程序中，开发者对某一时刻调度什么几乎无法控制，甚至完全无法控制；程序员只能创建线程，然后寄希望于底层操作系统能在可用的 CPU 上合理地调度它们。要构建一个对所有工作负载（workload）、在所有情况下都表现良好的通用调度程序（scheduler）是很难的，所以操作系统有时调度工作的方式并不理想。于是，我们有了……

关键问题：
如何不用线程构建并发服务器
怎样才能不用线程就构建出并发服务器（concurrent server），从而既保持对并发的控制，又避免一些似乎困扰着多线程应用程序的问题？

33.1 基本思想：事件循环

如上所述，我们要用的基本方法叫作**基于事件的并发**（event-based concurrency）。这种方法非常简单：只需等待某件事（即一个“事件”）发生；它发生时，检查它是什么类型的事件，然后做它所需的少量工作（可能包括发起 I/O 请求、安排其他事件以便将来处理，等等）。就这么简单！

在深入细节之前，我们先看看一个典型的基于事件的服务器是什么样子。这类应用程序都围绕一个简单的结构构建，这个结构叫作**事件循环**（event loop）。事件循环的伪代码（pseudocode）如下：

```
while (1) {
    events = getEvents();
    for (e in events)
        processEvent(e);
}
```

真的就这么简单。主循环只是调用 `getEvents()` 等着有事可做，然后逐一处理返回的每个事件；处理每个事件的代码叫作**事件处理程序**（event handler）。重要的是，当一个处理程序处理某个事件时，它是系统中唯一正在进行的活动；因此，决定下一个处理哪个事件，就相当于调度。这种对调度的显式控制，是基于事件的方法的根本优势之一。

但这番讨论给我们留下了一个更大的问题：基于事件的服务器究竟如何确定正在发生哪些事件，尤其是网络和磁盘 I/O 方面的事件？具体来说，事件服务器怎么知道有发给它的消息到达了？

33.2 一个重要的 API：`select()`（或 `poll()`）

有了这个基本的事件循环，接下来必须解决如何接收事件的问题。大多数系统都通过 `select()` 或 `poll()` 系统调用（system call）提供了一个基本的 API。

这些接口让程序能做的事情很简单：检查有没有需要处理的传入 I/O。例如，设想一个网络应用程序（比如 Web 服务器）想检查是否有网络数据包（packet）到达，以便为它们提供服务。这两个系统调用正好可以做到这一点。

以 `select()` 为例。（Mac 上的）手册页（manual page）这样描述这个 API：

```
int select(int nfds,
           fd_set *restrict readfds,
           fd_set *restrict writefds,
           fd_set *restrict errorfds,
           struct timeval *restrict timeout);
```

手册页上的实际描述是：*`select()` 检查地址通过 `readfds`、`writefds` 和 `errorfds` 传入的 I/O 描述符集合，分别看其中是否有描述符已准备好读、已准备好写，或者有待处理的异常情况。每个集合只检查前 `nfds` 个描述符，即检查描述符集合中从 0 到 `nfds-1` 的描述符。返回时，`select()` 会把给定的描述符集合替换为子集，子集由已准备好执行所请求操作的那些描述符组成。`select()` 返回所有集合中就绪描述符的总数。*

关于 `select()`，有几点要说明。首先，注意它既能检查描述符是否可以读，也能检查是否可以写；前者让服务器能确定有新的数据包到达、需要处理，后者让服务器知道什么时候可以回复（即发送队列（queue）未满）。

其次，注意 `timeout` 参数。一种常见用法是把超时（timeout）设为 NULL，这会让 `select()` 无限期地阻塞，直到某个描述符就绪（ready）。不过，更健壮的服务器通常会指定某种超时；一种常见的做法是把超时设为零，这样对 `select()` 的调用就会立即返回。

`poll()` 系统调用与它非常相似。详情请参阅它的手册页，或者 Stevens 和 Rago 的书 [SR05]。

补充：阻塞接口与非阻塞接口
阻塞（blocking）接口，或者说**同步**（synchronous）接口，在返回调用者之前会完成全部工作；非阻塞（non-blocking）接口，或者说**异步**（asynchronous）接口，则开始一些工作后立即返回，让需要完成的工作在后台进行。
阻塞调用的罪魁祸首通常是某种 I/O。例如，如果一个调用必须从磁盘读取数据才能完成，它就可能阻塞，等待已发往磁盘的 I/O 请求返回。
非阻塞接口可以用在任何风格的编程中，例如使用线程的编程；但在基于事件的方法中必不可少，因为一个阻塞的调用会让一切都停滞不前。

无论用哪一个，这些基本原语都为我们提供了一种方法，可以构建一个非阻塞的事件循环：它只需检查有没有传入的数据包，从有消息到达的套接字（socket）中读取数据，并根据需要回复。

33.3 使用 `select()`

为了更具体一些，我们来看看如何用 `select()` 查看哪些网络描述符上有传入的消息。图 33.1 给出了一个简单的例子。

这段代码其实相当好懂。完成一些初始化之后，服务器进入一个无限循环。在循环中，它先用 `FD_ZERO()` 宏清空文件描述符（file descriptor）集合，再用 `FD_SET()` 把从 `minFD` 到 `maxFD` 的所有文件描述符都加入集合。例如，这个描述符集合可能代表服务器关注的所有网络套接字。最后，服务器调用 `select()`，查看哪些连接上有可用的数据。接着在循环中使用 `FD_ISSET()`，事件服务器就能看到哪些描述符已经有数据就绪，并处理传入的数据。

当然，真正的服务器会比这复杂得多，还需要发送消息、发起磁盘 I/O 以及处理许多其他细节的逻辑。API 方面的更多信息请参阅 Stevens 和 Rago 的书 [SR05]；Pai 等人和 Welsh 等人的论文则很好地概述了基于事件的服务器的一般流程 [PDZ99, WCB01]。

33.4 为什么更简单？不需要锁

在单个 CPU 上运行基于事件的应用程序时，并发程序（concurrent program）中的那些问题就不复存在了。具体来说，由于一次只处理一个事件，所以不需要获取或释放锁（lock）；基于事件的服务器不会被另一个线程中断（interrupted），因为它明确就是单线程（single-threaded）的。因此，多线程程序中常见的并发 bug（concurrency bug），不会出现在基本的基于事件的方法中。

33.5 一个问题：阻塞的系统调用

到目前为止，基于事件的编程听起来很棒，对吧？你只需编写一个简单的循环，在事件出现时处理它们。你甚至都不用考虑加锁（locking）！但有一个问题：如果某个事件要求你发起一个可能阻塞的系统调用，该怎么办？

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <sys/types.h>
#include <unistd.h>

int main(void) {
  // 打开并设置好一批套接字（未列出）
  // 主循环
  while (1) {
    // 把 fd_set 初始化为全零
    fd_set readFDs;
    FD_ZERO(&readFDs);

    // 现在为这个服务器关心的
    // 描述符设置相应的位
    //  （为简单起见，设置从 min 到 max 的全部描述符）
    int fd;
    for (fd = minFD; fd < maxFD; fd++)
      FD_SET(fd, &readFDs);

    // 执行 select
    int rc = select(maxFD+1, &readFDs, NULL, NULL, NULL);

    // 用 FD_ISSET() 检查哪些描述符确实有数据
    int fd;
    for (fd = minFD; fd < maxFD; fd++)
      if (FD_ISSET(fd, &readFDs))
        processFD(fd);
  }
}
```

图 33.1：**使用 `select()` 的简单代码**

例如，设想客户端向服务器发来一个请求，要求从磁盘读取一个文件，并把内容返回给发出请求的客户端（很像一个简单的 HTTP 请求）。为了处理这样的请求，某个事件处理程序最终必须发起 `open()` 系统调用来打开文件，接着发起一系列 `read()` 调用来读取文件。文件读入内存后，服务器很可能会开始把结果发送给客户端。

`open()` 和 `read()` 调用都可能向存储系统发出 I/O 请求（当所需的元数据（metadata）或数据还不在内存中时），因此可能要花很长时间才能完成。基于线程的服务器不存在这个

提示：不要在基于事件的服务器中阻塞

基于事件的服务器能细粒度（fine-grained）地控制任务调度。不过，要保持这种控制，就绝不能发起任何会阻塞调用者执行的调用；不遵守这条设计提示，结果就是服务器被阻塞、客户端很恼火，而且别人会严重怀疑你到底有没有读过本书的这一部分。

 © 2008–23, ARPACI-DUSSEAU

问题：发出 I/O 请求的线程挂起（等待 I/O 完成）时，其他线程可以运行，从而让服务器继续推进。事实上，I/O 和其他计算之间这种自然的**重叠**（overlap），正是基于线程的编程显得自然而直接的原因。

然而，在基于事件的方法中，没有其他线程可以运行，只有主事件循环。这意味着，如果某个事件处理程序发起了一个会阻塞的调用，整个服务器就会跟着阻塞，直到这个调用完成。事件循环阻塞时，系统就闲着，这可能造成巨大的资源浪费。因此，在基于事件的系统中有一条必须遵守的规则：不允许阻塞调用。

33.6 一个解决方案：异步 I/O

为了突破这个限制，许多现代操作系统引入了向磁盘系统发出 I/O 请求的新方法，统称为**异步 I/O**（asynchronous I/O）。这些接口让应用程序可以发出 I/O 请求，并在 I/O 完成之前立即把控制权返回给调用者；另外还有一些接口，让应用程序能够确定各个 I/O 是否已经完成。

例如，我们来看看 Mac 上提供的接口（其他系统也有类似的 API）。这些 API 围绕一个基本结构展开，即 `struct aiocb`，通常称为 **AIO 控制块**（AIO control block）。这个结构的简化版本如下，更多信息请参阅手册页：

```
struct aiocb {
    int              aio_fildes;    // 文件描述符
    off_t            aio_offset;    // 文件内的偏移量
    volatile void    *aio_buf;      // 缓冲区的位置
    size_t           aio_nbytes;    // 传输的长度
};
```

要对一个文件发起异步读取，应用程序应该先在这个结构中填入相关信息：要读取的文件的文件描述符 `aio_fildes`、文件内的偏移量（offset）`aio_offset`、请求的长度 `aio_nbytes`，以及读取结果应复制到的目标内存位置 `aio_buf`。

填好这个结构后，应用程序必须发起读取文件的异步调用；在 Mac 上，这个 API 就是**异步读**（asynchronous read）API：

```
int aio_read(struct aiocb *aiocbp);
```

这个调用会尝试发起 I/O；如果成功，它就立即返回，应用程序（即基于事件的服务器）可以继续自己的工作。

不过，这道谜题还剩最后一块要解决。我们怎么知道 I/O 什么时候完成，从而知道（`aio_buf` 指向的）缓冲区（buffer）里已经有了所请求的数据呢？

还需要最后一个 API。在 Mac 上，它（有点令人困惑地）叫作 `aio_error()`。这个 API 如下：

```
int aio_error(const struct aiocb *aiocbp);
```

这个系统调用检查 `aiocbp` 所指的请求是否已经完成。如果已经完成，该例程返回成功（用零表示）；如果没有，就返回 `EINPROGRESS`。因此，对于每个尚未完成的异步 I/O

 © 2008–23, ARPACI-DUSSEAU

（asynchronous I/O），应用程序可以调用 `aio_error()`，定期**轮询**（poll）系统，以确定该 I/O 是否已经完成。

你可能已经注意到，检查 I/O 是否完成是一件很痛苦的事；如果某个时刻程序已经发出了几十甚至上百个 I/O，它是应该不停地反复检查每一个，还是先等一会儿，还是……？

为了解决这个问题，有些系统提供了一种基于**中断**（interrupt）的方法。这种方法利用 UNIX **信号**（signal），在异步 I/O（asynchronous I/O）完成时通知应用程序，从而无需反复询问系统。轮询与中断的问题在设备中也会出现，你会在 I/O 设备一章中看到（或者已经看到了）。

在没有异步 I/O（asynchronous I/O）的系统中，就无法实现纯粹的基于事件的方法。不过，聪明的研究人员已经找到了一些效果相当不错的替代方法。例如，Pai 等人 [PDZ99] 描述了一种混合方法：用事件处理网络数据包，用线程池（thread pool）管理尚未完成的 I/O。详情请阅读他们的论文。

33.7 另一个问题：状态管理

基于事件的方法还有一个问题：这类代码通常比传统的基于线程的代码更难写。原因如下：事件处理程序发起异步 I/O（asynchronous I/O）时，必须把一些程序状态打包起来，供 I/O 最终完成时下一个事件处理程序使用；而基于线程的程序不需要这些额外的工作，因为程序需要的状态就在线程的栈（stack）上。Adya 等人把这项工作称为**手动栈管理**（manual stack management），它是基于事件的编程所固有的 [A+02]。

为了把这一点说得更具体，我们来看一个简单的例子：一个基于线程的服务器需要从一个文件描述符 `fd` 读取数据，读完后，把从文件读到的数据写到一个网络套接字描述符（socket descriptor）`sd`。代码（忽略错误检查）如下：

```
int rc = read(fd, buffer, size);
rc = write(sd, buffer, size);
```

可以看到，在多线程程序中做这种工作轻而易举；`read()` 最终返回时，代码马上就知道要写到哪个套接字，因为这个信息就在线程的栈上（在变量 `sd` 中）。

在基于事件的系统中，事情就没那么简单了。要完成同样的任务，我们首先要用上面介绍的 AIO 调用，以异步方式发起读取。假设我们随后用 `aio_error()` 调用定期检查读取是否完成；当这个调用告诉我们读取已经完成时，基于事件的服务器怎么知道接下来该做什么呢？

正如 Adya 等人 [A+02] 所述，解决办法是使用一种古老的编程语言结构，叫作**延续**（continuation）[FHK84]。它听起来很复杂，但思路其实相当简单：基本上就是把完成这个事件的处理所需的信息记录在某个数据结构（data structure）中；当事件发生时（即磁盘 I/O 完成时），查出所需的信息，然后处理这个事件。

在这个具体的例子中，解决办法是把套接字描述符（socket descriptor）`sd` 记录在某种数据结构中，例如散列表（hash table），以文件描述符 `fd` 为索引。磁盘 I/O 完成时，事件处理程序用文件描述符查找这个延续，得到套接字描述符（socket descriptor）的值并返回给调用者。这时（终于），服务器就可以完成最后一点工作，把数据写到套接字上。

 © 2008–23, ARPACI-DUSSEAU

补充：UNIX 信号

在所有现代 UNIX 变体中，都有一套庞大而有趣的基础设施，叫作**信号**（signal）。最简单地说，信号提供了一种与进程（process）通信的方式。具体来说，信号可以投递给应用程序；投递信号时，应用程序会停下手头正在做的事，转去运行**信号处理程序**（signal handler），也就是应用程序中处理该信号的某段代码。处理完后，进程就恢复之前的行为。

每个信号都有一个名字，比如 **HUP**（挂起，hang up）、**INT**（中断，interrupt）、**SEGV**（段违规，segmentation violation）等；详情请参阅手册页。有意思的是，有时是内核自己发出信号。例如，当你的程序遇到段违规（segmentation violation）时，操作系统会向它发送一个 **SIGSEGV**（给信号名加上前缀 **SIG** 是很常见的做法）；如果你的程序配置为捕获这个信号，你实际上可以运行一些代码来响应这种错误的程序行为（这对调试很有帮助）。如果一个信号发送给了没有配置处理它的进程，就会执行默认行为；对于 SEGV，进程会被杀死。

下面是一个简单的程序，它进入一个无限循环，但在此之前设置了一个信号处理程序来捕获 SIGHUP：

```
void handle(int arg) {
    printf("stop wakin' me up...\n");
}

int main(int argc, char *argv[]) {
    signal(SIGHUP, handle);
    while (1)
        ; // 啥也不干，就接几个信号玩玩
    return 0;
}
```

你可以用命令行工具 **kill**（没错，这个名字既古怪又带点攻击性）向它发送信号。这样做会打断程序中的主 while 循环，转去运行处理程序代码 `handle()`：

```
prompt> ./main &
[3] 36705
prompt> kill -HUP 36705
stop wakin' me up...
prompt> kill -HUP 36705
stop wakin' me up...
```

关于信号，要学的东西还有很多，一章都远远讲不完，更别说一页了。和往常一样，有一个很好的资料来源：Stevens 和 Rago 的书 [SR05]。有兴趣的话可以深入阅读。

33.8 使用事件时仍然存在的难题

基于事件的方法还有其他几个难点，我们也应该提一下。例如，当系统从单 CPU 发展到多 CPU 时，基于事件的方法就失去了一部分简洁性。具体来说，为了利用多个 CPU，事

件服务器必须并行地（in parallel）运行多个事件处理程序；这样一来，常见的同步问题就会出现，比如临界区（critical section），也必须采用常见的解决办法，比如锁。因此，在现代多核（multicore）系统上，已经不可能再做那种不需要锁的简单事件处理了。

基于事件的方法的另一个问题是，它不能很好地与某些系统活动结合，比如**分页**（paging）。例如，如果一个事件处理程序发生了缺页异常（page fault），它就会阻塞，于是在缺页异常处理完之前，服务器都无法继续推进。尽管服务器的结构已经设计成避免显式阻塞，但这种由缺页异常引起的隐式阻塞很难避免，一旦频繁出现，就可能导致严重的性能问题。

第三个问题是，随着各种例程的确切语义发生变化，基于事件的代码可能会越来越难以管理 [A+02]。例如，如果一个例程从非阻塞变成了阻塞，那么调用这个例程的事件处理程序也必须随之改变，把自己拆成两部分，以适应这个例程的新特性。由于阻塞对基于事件的服务器来说是灾难性的，程序员必须时刻留意每个事件所用的 API 在语义上的这类变化。

最后，虽然现在大多数平台都已经支持异步磁盘 I/O，但实现这一点花了很长时间 [PDZ99]，而且它与异步网络 I/O 的结合，始终不像你想象的那样简单统一。例如，人们本希望只用 `select()` 接口来管理所有尚未完成的 I/O，但通常需要把用于网络的 `select()` 和用于磁盘 I/O 的 AIO 调用组合起来使用。

33.9 小结

我们简要介绍了另一种并发风格：基于事件的并发（event-based concurrency）。基于事件的服务器把调度的控制权交给应用程序本身，但代价是一定的复杂性，而且难以与现代系统的其他方面结合，例如分页。由于存在这些挑战，还没有哪一种方法脱颖而出成为最佳方案；因此，在未来许多年里，线程和事件很可能会作为解决同一并发问题的两种不同方法而长期并存。想了解更多，可以读一些研究论文（例如 [A+02, PDZ99, vB+03, WCB01]），或者更好的办法是，写一些基于事件的代码。

参考文献

[A+02] “Cooperative Task Management Without Manual Stack Management” by Atul Adya, Jon Howell, Marvin Theimer, William J. Bolosky, John R. Douceur. USENIX ATC ’02, Monterey, CA, June 2002. *This gem of a paper is the first to clearly articulate some of the difficulties of event-based concurrency, and suggests some simple solutions, as well explores the even crazier idea of combining the two types of concurrency management into a single application!*

[FHK84] “Programming With Continuations” by Daniel P. Friedman, Christopher T. Haynes, Eugene E. Kohlbecker. In Program Transformation and Programming Environments, Springer Verlag, 1984. *The classic reference to this old idea from the world of programming languages. Now increasingly popular in some modern languages.*

[N13] “Node.js Documentation” by the folks who built node.js. Available: nodejs.org/api. *One of the many cool new frameworks that help you readily build web services and applications. Every modern systems hacker should be proficient in frameworks such as this one (and likely, more than one). Spend the time and do some development in one of these worlds and become an expert.*

[O96] “Why Threads Are A Bad Idea (for most purposes)” by John Ousterhout. Invited Talk at USENIX ’96, San Diego, CA, January 1996. *A great talk about how threads aren’t a great match for GUI-based applications (but the ideas are more general). Ousterhout formed many of these opinions while he was developing Tcl/Tk, a cool scripting language and toolkit that made it 100x easier to develop GUI-based applications than the state of the art at the time. While the Tk GUI toolkit lives on (in Python for example), Tcl seems to be slowly dying (unfortunately).*

[PDZ99] “Flash: An Efficient and Portable Web Server” by Vivek S. Pai, Peter Druschel, Willy Zwaenepoel. USENIX ’99, Monterey, CA, June 1999. *A pioneering paper on how to structure web servers in the then-burgeoning Internet era. Read it to understand the basics as well as to see the authors’ ideas on how to build hybrids when support for asynchronous I/O is lacking.*

[SR05] “Advanced Programming in the UNIX Environment” by W. Richard Stevens and Stephen A. Rago. Addison-Wesley, 2005. *Once again, we refer to the classic must-have-on-your-bookshelf book of UNIX systems programming. If there is some detail you need to know, it is in here.*

[vB+03] “Capriccio: Scalable Threads for Internet Services” by Rob von Behren, Jeremy Condit, Feng Zhou, George C. Necula, Eric Brewer. SOSP ’03, Lake George, New York, October 2003. *A paper about how to make threads work at extreme scale; a counter to all the event-based work ongoing at the time.*

[WCB01] “SEDA: An Architecture for Well-Conditioned, Scalable Internet Services” by Matt Welsh, David Culler, and Eric Brewer. SOSP ’01, Banff, Canada, October 2001. *A nice twist on event-based serving that combines threads, queues, and event-based handling into one streamlined whole. Some of these ideas have found their way into the infrastructures of companies such as Google, Amazon, and elsewhere.*

作业（编程）

在这个（简短的）作业中，你会积累一些编写基于事件的代码的经验，并接触它的一些关键概念。祝你好运！

问题

1. 首先，编写一个简单的服务器，它能接受 TCP 连接并为其提供服务。如果你还不知道怎么做，得先上网查一查。让它一次只服务一个请求；每个请求都非常简单，比如获取当前时间。
2. 现在，加入 `select()` 接口。编写一个主程序，它能接受多个连接；再编写一个事件循环，检查哪些文件描述符上有数据，然后读取并处理这些请求。务必仔细测试，确保你正确地使用了 `select()`。
3. 接下来，让请求稍微有趣一点，模拟一个简单的 Web 服务器或文件服务器（file server）。每个请求都是读取某个文件（文件名在请求中给出）的内容，服务器的响应是：把文件读进缓冲区，然后把内容返回给客户端。用标准的 `open()`、`read()`、`close()` 系统调用实现这个功能。这里要小心一点：如果让它长时间运行，可能会有人想出办法，用它读取你电脑上的所有文件！
4. 现在改用本章介绍的异步 I/O（asynchronous I/O）接口，不再用标准的 I/O 系统调用。把异步接口融入你的程序有多难？
5. 为了好玩，给你的代码加上一些信号处理。信号的一种常见用途是“戳”一下服务器，让它重新加载某个配置文件，或者执行其他某种管理操作。一种比较自然的玩法是：给你的服务器加一个用户级文件缓存（cache），存放最近访问过的文件。实现一个信号处理程序，在服务器进程收到信号时清空这个缓存。
6. 最后是困难的部分：你怎么判断，花力气构建一个异步的、基于事件的方法是否值得？你能设计一个实验来展示它的好处吗？你的方法增加了多少实现复杂度？

 © 2008–23, ARPACI-DUSSEAU